

Track2Go Frontend Development Roadmap

Project Overview

Track2Go is a GitHub-like version control system with a React frontend and Express backend. The application features authentication, repository management, issue tracking, and real-time collaboration via Socket.IO.

UI/UX Expectations

Design Philosophy

- **Clean and Professional:** Mature, GitHub-inspired design - no flashy gradients or purple/blue glowing effects
 - **Light and Dark Mode:** All pages must support both themes with a toggle button
 - **Consistent Color Scheme:**
 - Light mode: White backgrounds, slate grays for text, slate-900 for primary actions
 - Dark mode: Slate-950 backgrounds, slate-900 cards, slate borders
 - **No Lucide React Icons:** Use custom SVG icons or simple HTML/CSS elements (except where explicitly allowed)
 - **Form Handling:** All forms must use `e.preventDefault()` at the top of handlers to prevent page reloads
 - **Component Structure:** Break down complex pages into reusable components (Navbar, Sidebar, Cards, etc.)
-

Backend API Endpoints

Authentication (`/api/auth`)

- `POST /signup` - Create new user account
- `POST /login` - User login
- `POST /logout` - User logout (requires auth)
- `POST /resetpassword` - Password reset (requires auth)

Users (`/api/user`)

- `GET /all` - Get all users
- `GET /:id` - Get user by ID
- `PATCH /:id` - Update user details

Repositories (`/api/repo`)

- `GET /` - Get current user's repositories (requires auth)
- `GET /all` - Get all repositories (requires auth)
- `POST /` - Create new repository (requires auth)
 - **Body:** `{ name, description, visibility: "Public"|"Private" }`
- `GET /by-name/:name` - Get repository by name (requires auth)
- `GET /:id` - Get repository by ID (requires auth)
- `PATCH /:id` - Update repository (requires auth)
- `DELETE /:id` - Delete repository (requires auth)

- PATCH `/:id/visibility` - Toggle repository visibility (requires auth)

Issues (`/api/repositories/:repoId/issues`)

- POST `/` - Create new issue (requires auth)
- GET `/` - Get all issues for repository (requires auth)
- GET `/by-label/:label` - Get issues by label (requires auth)
- GET `/by-title/:title` - Get issues by title (requires auth)
- GET `/:issueId` - Get issue by ID (requires auth)
- PATCH `/:issueId` - Update issue (requires auth)
- DELETE `/:issueId` - Delete issue (requires auth)

CLI Commands (Backend only, not frontend)

- init - Initialize repository
- add `<file>` - Add file to staging
- commit `<message>` - Commit changes
- push - Push to remote
- pull - Pull from remote
- revert `<commitHash>` - Revert to commit

Data Models

User

```
{
  _id: ObjectId,
  username: String (unique),
  email: String (unique),
  password: String,
  repositories: [ObjectId], // ref: Repository
  followedUsers: [ObjectId], // ref: User
  starredRepositories: [ObjectId] // ref: Repository (note: typo in backend)
}
```

Repository

```
{
  _id: ObjectId,
  owner: ObjectId, // ref: User
  name: String (unique),
  description: String,
  content: [String],
  visibility: "Public" | "Private",
  issues: [ObjectId] // ref: Issue
}
```

Issue

```
{
  _id: ObjectId,
  title: String,
  description: String,
  status: "open" | "closed",
  repository: ObjectId, // ref: Repository
  assignees: [ObjectId], // ref: User
}
```

```
labels: [String],
milestone: [String] // future implementation
}
```

Frontend Pages & Components

☒ Completed Pages

1. Signup Page (/signup)

Status: Complete **Features:**

- Username, email, password, confirm password fields
- Form validation (min 6 chars, password match)
- Password visibility toggles
- Dark mode support
- Error handling with alerts
- Navigation to login page **API:** POST /api/auth/signup

2. Login Page (/login)

Status: Complete **Features:**

- Email and password fields
- Password visibility toggle
- "Forgot password?" link
- Dark mode support
- Error handling
- Navigation to signup page **API:** POST /api/auth/login

3. Dashboard (/)

Status: Complete **Features:**

- Navbar with search, dark mode toggle, "New" button, user avatar
- Sidebar with tabs: Repositories, Starred, Following (with counts)
- Repository cards showing owner/name, description, language, last updated
- Empty states for no data
- Loading spinner
- Dark mode support **API:** Uses `getUser()` helper which calls GET /api/user/\${userId} (userId from localStorage) **Components:** Navbar, Sidebar, RepositoryCard

4. New Repository Page (/repo/new)

Status: Complete **Features:**

- Repository name input (required, min 3 chars, alphanumeric/hyphens/underscores only)
- Description input (optional)
- Visibility selector (Public/Private) with visual radio buttons
- Random name generator
- Form validation
- Dark mode support
- Back to dashboard button **API:** POST /api/repo **Navigation:** Redirects to dashboard on success

🔑 Pages to Build

5. Repository Details Page (/repo/:id or /repo/:name)

Priority: High Features:

- Repository header: name, visibility badge, star/fork counts
- Tabs: Code, Issues, Settings
- File browser (if content exists)
- README viewer
- Clone/download buttons
- Dark mode support **API:**
- GET /api/repo/:id or GET /api/repo/by-name/:name **Components:** RepoHeader, FileBrowser, ReadmeViewer

6. Issues Page (/repo/:id/issues)

Priority: High Features:

- List of issues with title, status (open/closed), labels, assignees
- Filter by label, status
- Search by title
- "New Issue" button
- Dark mode support **API:**
- GET /api/repositories/:repoId/issues
- GET /api/repositories/:repoId/issues/by-label/:label **Components:** IssueList, IssueCard, FilterBar

7. New/Edit Issue Page (/repo/:id/issues/new or /repo/:id/issues/:issueId/edit)

Priority: High Features:

- Title input (required)
- Description textarea
- Labels input (comma-separated or tags)
- Assignees selector (dropdown of users)
- Status selector (open/closed)
- Dark mode support **API:**
- POST /api/repositories/:repoId/issues (create)
- GET /api/repositories/:repoId/issues/:issueId (fetch for edit)
- PATCH /api/repositories/:repoId/issues/:issueId (update)

8. Issue Details Page (/repo/:id/issues/:issueId)

Priority: Medium Features:

- Full issue display with title, description, status, labels, assignees
- Comment section (if comments are implemented in backend)
- Edit/Delete buttons (for issue owner)
- Status toggle (open/close)
- Dark mode support **API:**
- GET /api/repositories/:repoId/issues/:issueId
- PATCH /api/repositories/:repoId/issues/:issueId

- DELETE /api/repositories/:repoId/issues/:issueId

9. User Profile Page (/user/:id)

Priority: Medium Features:

- User info: username, email, avatar
- User's repositories
- Starred repositories
- Following/followers counts
- Follow/Unfollow button (if API exists)
- Dark mode support **API:**
- GET /api/user/:id
- Need: POST /api/user/:id/follow (not in current routes)

10. Explore/All Repositories Page (/explore)

Priority: Medium Features:

- Grid/list of all public repositories
- Search and filter
- Dark mode support **API:**
- GET /api/repo/all

11. Settings Page (/settings)

Priority: Low Features:

- Update user details (email, username)
- Password reset
- Dark mode preference **API:**
- PATCH /api/user/:id
- POST /api/auth/resetpassword

Technical Implementation Notes

Authentication & Storage

- Store `userId` in `localStorage` after login
- Store JWT token in `localStorage` or cookies
- Axios interceptor should add `Authorization: Bearer <token>` header to all requests
- Base URL for axios: `http://localhost:3000/api/` (configured globally)

Routing Structure

```
/signup - Signup page
/login - Login page
/ - Dashboard (requires auth)
/repo/new - New repository page (requires auth)
/repo/:id - Repository details (requires auth)
/repo/:id/issues - Issues list (requires auth)
/repo/:id/issues/new - New issue (requires auth)
/repo/:id/issues/:issueId - Issue details (requires auth)
/user/:id - User profile (requires auth)
```

/explore - All repositories (requires auth)
/settings - User settings (requires auth)

Socket.IO Integration

- Socket server is running on the same port as Express
- `joinRoom(userId)` event to join user-specific room
- Can be used for real-time notifications, live collaboration
- Frontend should connect to socket and emit `joinRoom` with `userId`

Dark Mode Implementation

- Use React state: `const [darkMode, setDarkMode] = useState(false)`
- Apply conditional Tailwind classes: `${darkMode ? 'bg-slate-950' : 'bg-slate-50'}`
- Consider persisting preference in `localStorage`

Form Validation Patterns

```
const handleSubmit = async (e) => {  
  e.preventDefault() // MUST be first line  
  
  // Validation  
  if (!form.field) {  
    return setError('Field is required')  
  }  
  
  try {  
    setLoading(true)  
    setError('')  
    await axios.post('/endpoint', form)  
    navigate('/success-page')  
  } catch (err) {  
    setError(err.response?.data?.message || 'Operation failed')  
  } finally {  
    setLoading(false)  
  }  
}
```

Missing Backend APIs (May Need)

1. **Star/Unstar Repository:** POST `/api/repo/:id/star`, DELETE `/api/repo/:id/star`
 2. **Follow/Unfollow User:** POST `/api/user/:id/follow`, DELETE `/api/user/:id/follow`
 3. **Fork Repository:** POST `/api/repo/:id/fork`
 4. **Comments on Issues:** CRUD operations for comments (if needed)
-

Development Priority Order

1. ☒ Authentication (Signup/Login)
2. ☒ Dashboard with repository list
3. ☒ New Repository page
4.  Repository Details page
5.  Issues List page
6.  New/Edit Issue page

7. Issue Details page
 8. User Profile page
 9. Explore page
 10. Settings page
-

Component Library

Using **shadcn/ui** components:

- Button
- Input
- Alert, AlertDescription
- Card, CardContent, CardHeader, CardTitle (if needed)

Lucide-react icons can be used where appropriate for better icon quality.